

More lambda calculus

Exercise sheet 6 (solutions)

Exercise 6.1. Define the following functions for pairs:

- (a) SWAP, which exchanges a pair's components;
- (b) MAPPAIR, which receives f , g and a pair (x, y) , and returns $(f x, g y)$.

Solution. SWAP rebuilds the pair with its second component first. MAPPAIR applies f to the first component and g to the second, then pairs the results.

$$\begin{aligned}\text{SWAP} &= \lambda p. \text{PAIR} (\text{SND } p) (\text{FST } p), \\ \text{MAPPAIR} &= \lambda f. \lambda g. \lambda p. \text{PAIR} (f (\text{FST } p)) (g (\text{SND } p)).\end{aligned}$$

□

Exercise 6.2. Define the following functions for Scott lists. You may use Θ and the concatenation function APP from the lecture.

- (a) ISEEMPTY, which returns TRUE on the empty list and FALSE otherwise;
- (b) HEAD, which returns the first element or NIL on an empty list;
- (c) LEN, which returns the length as a Church numeral;
- (d) MAP, which applies a function to each element of a list;
- (e) REV, which reverses a list.

Solution. ISEEMPTY returns TRUE in the empty case and ignores the head and tail to return FALSE otherwise. HEAD returns NIL in the empty case and selects the head otherwise:

$$\begin{aligned}\text{ISEEMPTY} &= \lambda l. l \text{ TRUE } (\lambda h. \lambda t. \text{FALSE}), \\ \text{HEAD} &= \lambda l. l \text{ NIL } (\lambda h. \lambda t. h).\end{aligned}$$

For LEN, return zero on the empty list; otherwise increment the recursively computed length of the tail:

$$\text{LEN} = \Theta(\lambda r. \lambda l. l \bar{0} (\lambda h. \lambda t. \text{SUCC}(r t))).$$

For MAP, transform the head with f and recursively map the tail; the empty list stays empty:

$$\text{MAP} = \lambda f. \Theta(\lambda r. \lambda l. l \text{ NIL } (\lambda h. \lambda t. \text{CONS} (f h) (r t))).$$

For REV, append the head as a singleton after the recursively reversed tail; the empty list stays empty:

$$\text{REV} = \Theta(\lambda r. \lambda l. l \text{ NIL } (\lambda h. \lambda t. \text{APP} (r t) (\text{CONS } h \text{ NIL}))).$$

□

A **Church list** $[a_1, \dots, a_n]$ is represented by the term $\lambda z. \lambda c. c a_1 (c a_2 (\dots (c a_n z) \dots))$. It takes an *empty-case value* z , then a *combining function* c . In particular, the empty list is $\text{NIL} = \lambda z. \lambda c. z$ (as for Scott lists, but with a different meaning). A new element (*head*) is prepended to a list via the *constructor* $\text{CLIST} = \lambda h. \lambda l. \lambda z. \lambda c. c h (l z c)$.

Exercise 6.3. * Define the functions for Church lists without using recursion:

- (a) SUM and PROD, which return the sum resp. the product of elements, for a list of numerals;
- (b) LEN, which returns the length as a Church numeral;
- (c) APP, which concatenates two lists;
- (d) MAP, which applies a function to each element of a list;
- (e) REV, which reverses a list.

Solution. Here all operations use the Church encoding. For SUM and PROD, combine the elements by addition or multiplication, using 0 or 1 for the empty list:

$$\text{SUM} = \lambda l. l \bar{0} \text{ADD}, \quad \text{PROD} = \lambda l. l \bar{1} \text{MULT}.$$

For LEN, the combining function ignores the head and increments the length already computed for the remaining list:

$$\text{LEN} = \lambda l. l \bar{0} (\lambda h. \lambda n. \text{SUCC } n).$$

For APP, rebuild the first list with CLIST, using the second list in place of its empty end:

$$\text{APP} = \lambda l_1. \lambda l_2. l_1 l_2 \text{CLIST}.$$

For MAP, transform each head with f and prepend it to the already transformed remainder:

$$\text{MAP} = \lambda f. \lambda l. l \text{NIL} (\lambda h. \lambda r. \text{CLIST } (f h) r).$$

For REV, append each head as a singleton after the already reversed remainder, using the Church-list APP above; the empty list stays empty:

$$\text{REV} = \lambda l. l \text{NIL} (\lambda h. \lambda r. \text{APP } r (\text{CLIST } h \text{NIL})).$$

□